

Wednesday 5:46 PM



lab_one.pdf
PDF

Here is my lab, run me though what we doing, what language, what is the task, how can i go about it



How do the files look from a glance, also why is there a
claude.md file?

Ari, in `src/app/page.tsx`, find the **Active tasks** section. You should currently have:

```
<section>
  <h2>Active tasks</h2>

  <form method="get">
```

Replace only that form-opening line with:

TypeScript

So it becomes:

```
<section>
  <h2>Active tasks</h2>

  <form method="get" className="sort-form">
    <label htmlFor="sort">Sort tasks by</label>

    <select
      id="sort"
      name="sort"
      defaultValue={sort}
    >
      <option value="dueDate">Due date</option>
      <option value="topic">Topic</option>
      <option value="status">Status</option>
    </select>

    <button type="submit">Apply sort</button>
  </form>

  {activeTasks.length === 0 ? (
```

Then in `src/app/globals.css` , add:

```
.sort-form {
  margin-bottom: 2px;
}
```

You are **replacing the existing** `<form method="get">` , not adding a second sorting form.



ive made my changes, ready for whats next

Worked for 1m 29s >

Ari, first lock in the overdue and layout work, then we'll start the automated tests.

1. Check and commit your current changes

Stop the server, then run:

```
npm run lint
npm run build
npm run start
```

If both complete successfully:

```
git status
git add .
git commit -m "Add overdue indicators and refine task layout"
git push
```

That commit should include:

- Derived overdue warnings
- Task-card styling
- Two-column create form
- Improved spacing

2. Install Vitest

The lab requires at least three deterministic tests, including one covering archiving or overdue behaviour. `lab_one`

Install Vitest:

PowerShell

Vitest's official setup uses `npm install -D vitest` and runs comfortably on your current Node version.  Vitest

3. Add the test command

Open `package.json` and find:

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
```

```
"lint": "eslint"  
}
```

Add the test script:

```
"scripts": {  
  "dev": "next dev",  
  "build": "next build",  
  "start": "next start",  
  "lint": "eslint",  
  "test": "vitest run"  
}
```

Be careful with the comma after the `lint` line.

4. Move the overdue rule into a testable file

Create:

```
src/lib/task-rules.ts
```

Add:

```
import { TaskStatus } from "../generated/prisma/client";  
  
export function isTaskOverdue(  
  dueDate: Date,  
  status: TaskStatus,  
  now: Date = new Date(),  
): boolean {  
  return (  
    dueDate.getTime() < now.getTime() &&  
    status !== TaskStatus.COMPLETE  
  );  
}
```

The optional `now` argument allows tests to use a fixed time rather than relying on the real clock.

5. Update `src/app/page.tsx`

Add this import:

TypeScript

Remove the old local function from `page.tsx` :

```
function isTaskOverdue(  
  dueDate: Date,  
  status: TaskStatus,  
) {  
  return (  
    dueDate.getTime() < Date.now() &&  
    status !== TaskStatus.COMPLETE  
  );  
}
```

If `TaskStatus` is no longer used anywhere else in `page.tsx`, change:

```
import {  
  Prisma,  
  TaskStatus,  
} from "@generated/prisma/client";
```

to:

TypeScript

Your existing call can remain unchanged:

```
const overdue = isTaskOverdue(  
  task.dueDate,  
  task.status,  
);
```

6. Create the first test file

Create a folder:

```
tests
```

Then create:

```
tests/task-rules.test.ts
```

Add:

```
import { describe, expect, it } from "vitest";
import { TaskStatus } from "../src/generated/prisma/client";
import { isTaskOverdue } from "../src/lib/task-rules";

describe("isTaskOverdue", () => {
  const now = new Date("2026-07-30T12:00:00.000Z");

  it("marks a past-due Todo task as overdue", () => {
    const dueDate = new Date("2026-07-29T12:00:00.000Z");

    expect(
      isTaskOverdue(dueDate, TaskStatus.TODO, now),
    ).toBe(true);
  });

  it("marks a past-due In-Progress task as overdue", () => {
    const dueDate = new Date("2026-07-29T12:00:00.000Z");

    expect(
      isTaskOverdue(
        dueDate,
        TaskStatus.IN_PROGRESS,
        now,
      ),
    ).toBe(true);
  });

  it("does not mark a completed task as overdue", () => {
    const dueDate = new Date("2026-07-29T12:00:00.000Z");

    expect(
      isTaskOverdue(
        dueDate,
        TaskStatus.COMPLETE,
        now,
      ),
    ).toBe(false);
  });

  it("does not mark a future task as overdue", () => {
    const dueDate = new Date("2026-07-31T12:00:00.000Z");

    expect(
      isTaskOverdue(dueDate, TaskStatus.TODO, now),
    ).toBe(false);
  });
});
```

```
});
});
```

7. Run the tests

PowerShell

You should see approximately:

```
✓ tests/task-rules.test.ts
  ✓ marks a past-due Todo task as overdue
  ✓ marks a past-due In-Progress task as overdue
  ✓ does not mark a completed task as overdue
  ✓ does not mark a future task as overdue
```

```
Test Files  1 passed
Tests       4 passed
```

These tests are deterministic because they use a fixed date. Do not commit this test setup yet—the next part is adding a **real archive test using a throwaway SQLite database**, which is the stronger requirement in the rubric.

    Sources

Branched from Branch · Full-stack Todo App

when i create the tests folder is that in src

No, Ari — create the `tests` folder in the **project root**, not inside `src`.

Your structure should look like:

```
coms3011a-lab1/
├─ prisma/
├─ public/
├─ src/
│  ├─ app/
│  ├─ generated/
│  └─ lib/
├─ tests/
│  └─ task-rules.test.ts
└─ package.json
```

```
├─ package-lock.json
└─ README.md
```

So in VS Code:

1. Right-click the main project folder `coms3011a-lab1`.
2. Select **New Folder**.
3. Name it `tests`.
4. Create `task-rules.test.ts` inside it.

The test imports should therefore use:

TypeScript

```
import { TaskStatus } from "../src/generated/prisma/client";
import { isTaskOverdue } from "../src/lib/task-rules";
```

Keeping tests at the root clearly separates application code from test code. The lab requires at least three real, deterministic tests runnable through one documented command.

lab_one

  ...  Sources

ChatGPT can make mistakes. Check important info.